

- 3, 5 years
- 1.5 y ofg
- 2y scalar

Keshav Sekharia

IIT BHU MNC 2022

WorldQuant (HFT) Quantitative Researcher

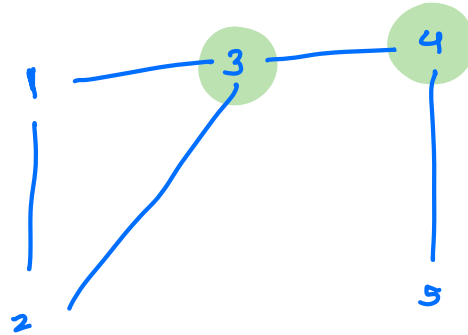
Google, Blr SWE III

CP module

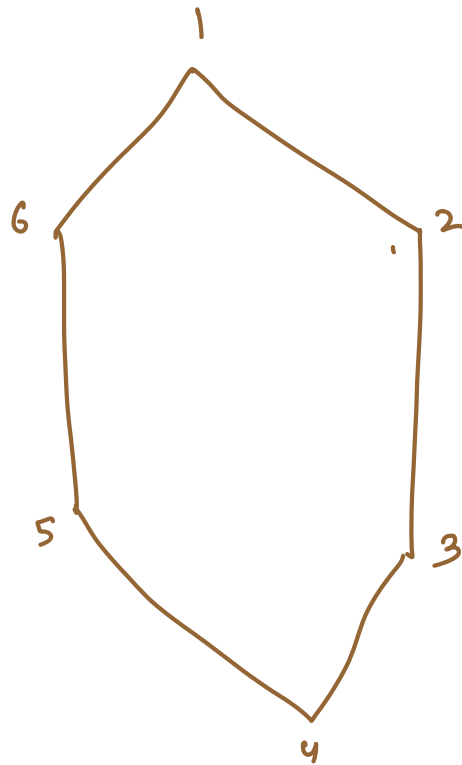
- AP & Bridges
- AVL Trees
- Binary Lifting + LCA
- Combinatorics
- Digit DP
- DP on Trees
- matrix Exponentiation
- Nim game
- Segment Tree (3)
- Tree Flattening

Articulation point & Bridges.

▷ AP



- Nodes whose removal will make the graph disconnected $\Rightarrow$ APs



○ APs

Q How do we identify Articulation points,

1. For all nodes:

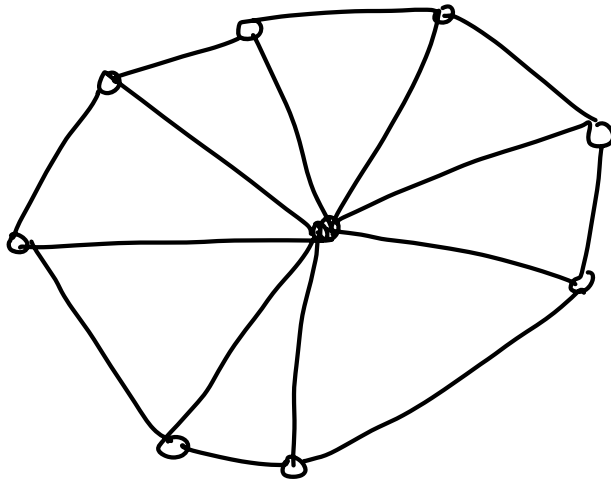
$O(V)$

$O(1)$ 1. Remove node

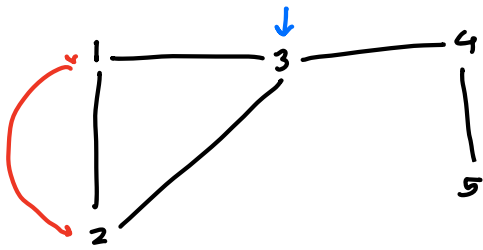
$O(V+E)$ 2. See if graph is connected or not

$O(1)$ 3. Add node

TC: $O(V \cdot (V+E))$

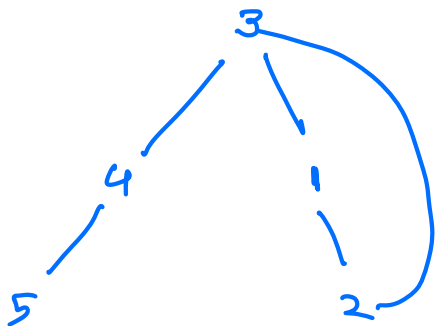
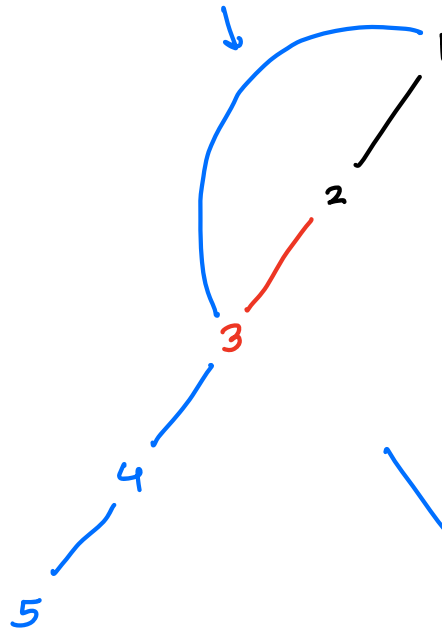


Back edges



Back edge

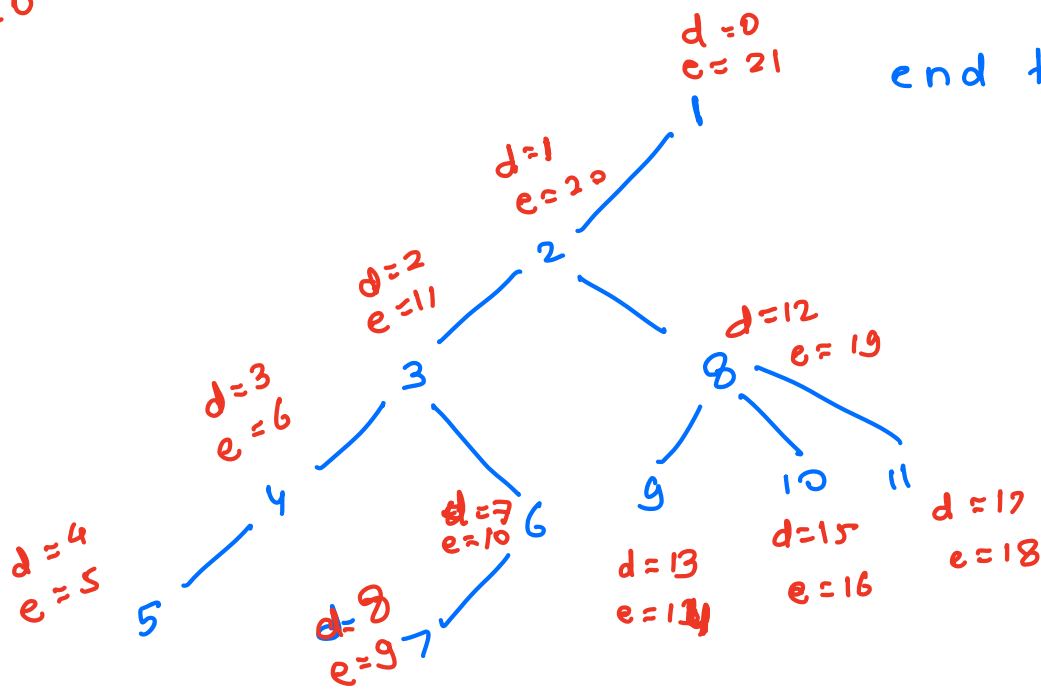
DFS



DFS
Tree

$t=0$

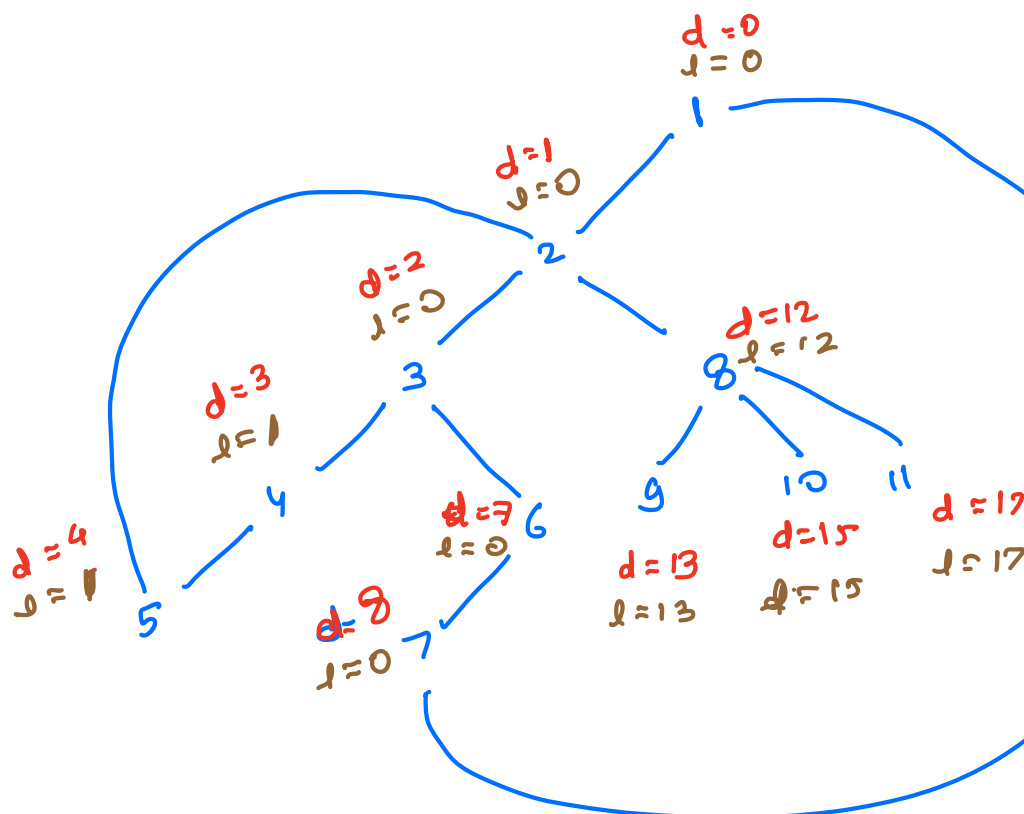
Discovery time
end time



low[time]



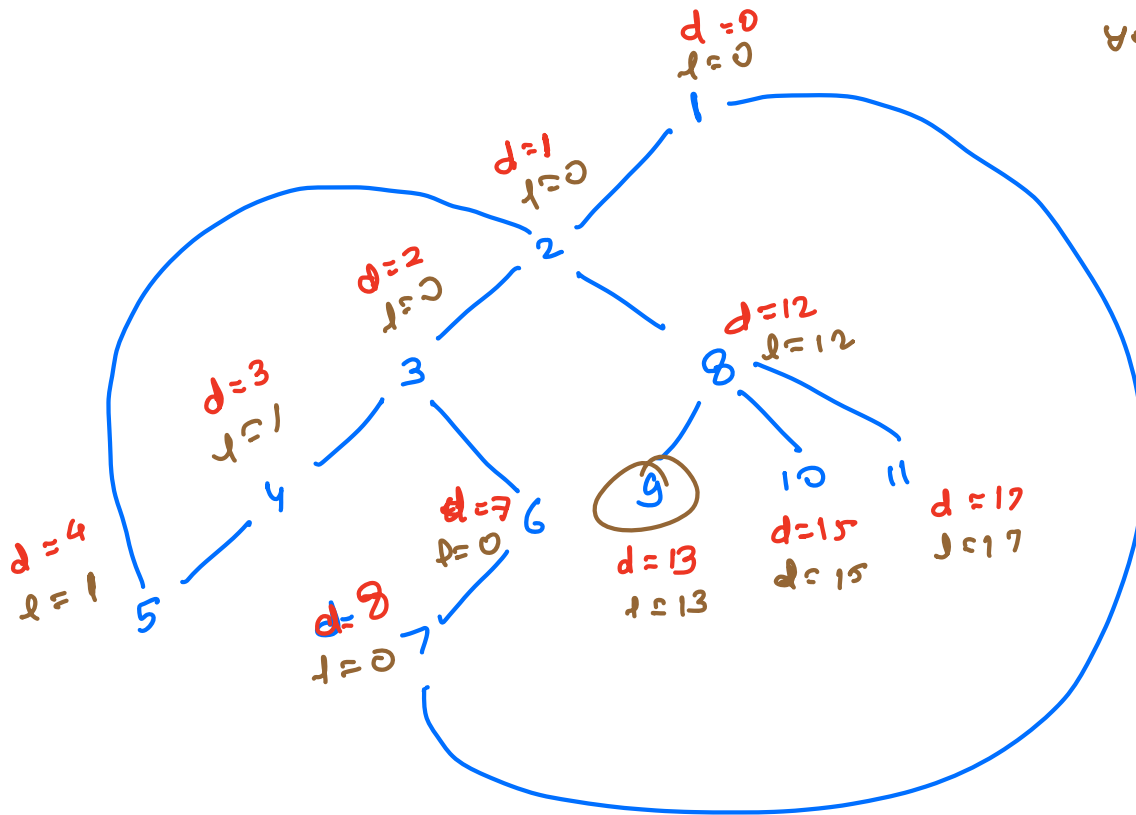
lowest discovery
time you or
your child
can reach,

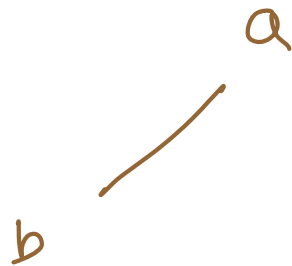
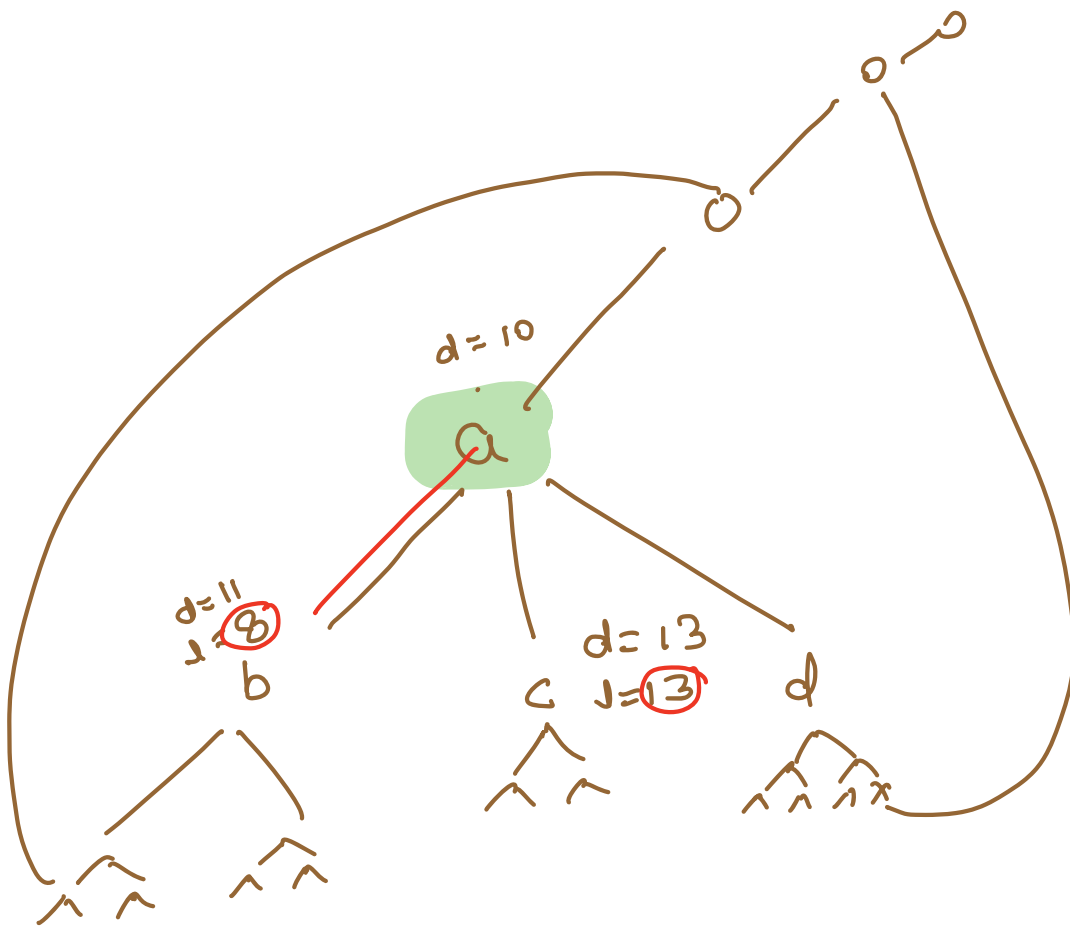


How [Hme]



lowest discovery
time you or
your child
connected to





$$\text{low}[b] < \text{dis}[a]$$

$$\text{low}[d] < \text{dis}[a]$$

$$\text{low}[c] > \text{dis}[a]$$

1.



if $(low[child] < dis[par])$

then child have a
back edge

if $(low[child] > dis[par])$

then child don't have a
back edge

2. if any of my child don't have
back edge, then I am AP.

3. if for any of my child:

if $(low[child] > dis[par])$

Then I am AP.

$t = 0$

void dfs (int j, int par, vis())

child = 0

vis[i] = True

dis[i] = t

low[i] = t

t++

for (j : adj[i])

if (j == par) continue;

if (vis[j] == False)

dfs (j, i)

child++

low[i] = min (low[j], low[i])

if (low[j] > dis[i])

{ isAP[i] = True

else

// Backedge

low[i] = min (low[i], dis[j])

Backedge

if (par == -1 && child > 1) isAP[i] = True

return

Tc: $O(V + E)$

Sc: $O(V + E)$

Q. Destructive mind

Graph

Q queries

→ node x ,

Tell if I remove x , the

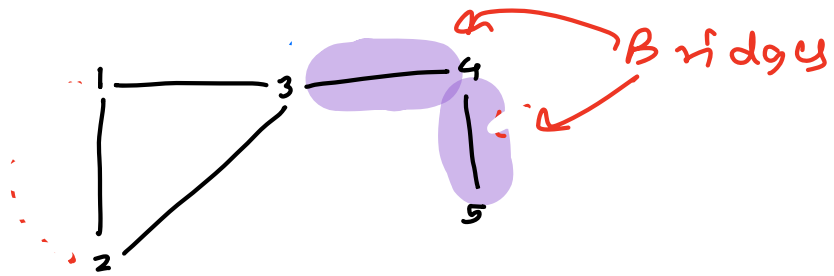
graph will be
disconnected or not.

ans : $isAP()$ ✓

if x is AP or not

—

1)



BF

1. Remove edge
2. Traversal
3. Add edge

$$TC: O(E(V+E))$$

```
void dfs ( int j, int par, vis[] )
```

```
    child = 0
```

```
    vis[i] = True
```

```
    disc[i] = t
```

```
    low[i] = t
```

```
    t++
```

```
    for ( j : adj[i] )
```

```
        if ( j == par ) continue;
```

```
        if ( vis[j] == False )
```

```
            dfs(j, i)
```

```
            low[i] = min ( low[j], low[i] )
```

```
            if ( low[j] > disc[i] )
```

```
                { print ( i to j is bridge )
```

```
            else
```

```
                // Backedge
```

```
                low[i] = min ( low[i], disc[j] )
```

```
    if ( par == -1 && child > 1 ) ISAP[i] = True
```

```
    return
```

child

child++

Backedge

codejenny.com

codechef.com